Code

```
def cost(state):
    conflicts = 0
    for i in range(4):
        for j in range(i + 1, 4):
            if abs(state[i] - state[j]) == abs(i - j):
                conflicts += 1
    return conflicts


def hill_climbing():
    current = [3, 1, 2, 0]

    while True:
        current_cost = cost(current)

        print("Current state:", current)
        print("Current cost:", current_cost)

        # Stop when the solution is found
        if current_cost == 0:
            print("Solution found:", current)
            break

        neighbors = []

        for i in range(4):
            for j in range(i + 1, 4):
                new_state = current.copy()
                new_state[i], new_state[j] = new_state[j], new_state[i]
                neighbors.append((cost(new_state), i, j, new_state))

        neighbors.sort(key=lambda x: (x[0], x[1], x[2]))

        print("Neighbor states:")

        for c, i, j, state in neighbors:
            print("Swap x%d and x%d:" % (i, j), state, "Cost =", c)

        best_cost, i, j, best_state = neighbors[0]

        if best_cost >= current_cost:
            print("No better neighbor found.")
            break

        print("Choose swap x%d and x%d" % (i, j))
        current = best_state
        print()


hill_climbing()
```

Output

```
Current state: [3, 1, 2, 0]
Current cost: 2
Neighbor states:
Swap x0 and x1: [1, 3, 2, 0] Cost = 1
Swap x0 and x2: [2, 1, 3, 0] Cost = 1
Swap x1 and x3: [3, 0, 2, 1] Cost = 1
Swap x2 and x3: [3, 1, 0, 2] Cost = 1
Swap x0 and x3: [0, 1, 2, 3] Cost = 6
Swap x1 and x2: [3, 2, 1, 0] Cost = 6
Choose swap x0 and x1

Current state: [1, 3, 2, 0]
Current cost: 1
Neighbor states:
Swap x2 and x3: [1, 3, 0, 2] Cost = 0
Swap x0 and x1: [3, 1, 2, 0] Cost = 2
Swap x0 and x2: [2, 3, 1, 0] Cost = 2
Swap x1 and x3: [1, 0, 2, 3] Cost = 2
Swap x0 and x3: [0, 3, 2, 1] Cost = 4
Swap x1 and x2: [1, 2, 3, 0] Cost = 4
Choose swap x2 and x3

Current state: [1, 3, 0, 2]
Current cost: 0
Solution found: [1, 3, 0, 2]
```